

Activitat 1.4

CRUD App

Content

1. Activity description.....	4
2. Technologies used.....	4
3. Prerequisites.....	4
4. Project structure.....	5
5. How to start the application.....	6
Useful commands.....	6
6. Using the web interface.....	6
7. REST API.....	7
8. How Docker is used in this project.....	8
Dockerfile.....	8
docker-compose.yml.....	8
9. Data persistence.....	8

1. Activity description

This activity is a small web application for managing a list of items. It lets you create, read, update and delete items (CRUD operations) through an API built with Express, and a simple web page that consumes that API from the browser.

The application runs inside a Docker container, so it works the same on any machine that has Docker installed, with no need to install Node.js.

Each item has two fields: a numeric identifier (id) and a name (name). The data is stored in a JSON file (data.json), so no database is required.

2. Technologies used

- **Node.js 18** as the runtime (official node:18 image).
- **Express** as the web server and API framework.
- **HTML and JavaScript** on the frontend, with no frameworks; fetch is used to call the API.
- **Docker and Docker Compose** to package and run the application.
- **data.json** as the data store.

3. Prerequisites

You only need the following installed:

- Git, to clone the repository.
- Docker with the Docker Compose plugin (the docker compose command).

You do not need to install Node.js or run npm install on your machine: dependencies are installed inside the Docker image.

4. Project structure

File / folder	Description
<code>server.js</code>	Express server. Defines the REST API (<code>/api/items</code> and <code>/api/reset</code>), serves the public folder and listens on port 3000.
<code>public/index.html</code>	Web page with the forms and buttons for each operation (read, create, search, update, delete and reset).
<code>public/script.js</code>	Browser-side code. Listens to the forms, calls the API with fetch and shows results and messages.
<code>data.json</code>	File where items are stored. Contains two sample items: Item A and Item B.
<code>package.json</code>	Defines the project, the start script (node server.js) and the express dependency.
<code>package-lock.json</code>	Pins the exact versions of the dependencies.
<code>Dockerfile</code>	Recipe for building the application image.
<code>docker-compose.yml</code>	Defines how the container runs: ports and volumes.
<code>.dockerignore</code>	Excludes <code>node_modules</code> when building the image.
<code>.gitignore</code>	Excludes <code>/node_modules</code> from the Git repository.
<code>README.md</code>	Repository front page (currently empty).

5. How to start the application

1. Clone the repository:

```
git clone https://github.com/carlesconesa34/ac4  
cd ac4
```

2. Build and start the container:

```
docker compose up -d --build
```

3. Open the application in your browser: <http://localhost:3000>

The terminal should print "CRUD app listening at <http://localhost:3000>". If you prefer to run it in the background, use `docker compose up --build -d`.

Useful commands

Command	What it does
<code>docker compose up --build</code>	Builds the image and starts the application.
<code>docker compose up -d</code>	Starts the application in the background.
<code>docker compose ps -a</code>	Shows the container status.
<code>docker compose down</code>	Stops and removes the container.
<code>docker compose down -v</code>	Same as above, and also deletes the volumes (useful if node_modules causes problems).

6. Using the web interface

The CRUD app page (<http://localhost:3000>) has one section for each operation. When it opens, the item list is loaded automatically.

Section	How to use it
Read all items	Click "Show items" to see every item with its ID and name.
Create an item	Type a name and click "Create". The item is created with a new ID and the list is refreshed.
Read one item	Enter an ID and click "Search". It shows that item, or an error if it does not exist.
Update an item	Enter the ID and the new name, then click "Update".
Delete an item	Enter the ID and click "Delete" to remove it.
Reset data	Click "Reset" to go back to the initial data (Item A and Item B).

Below the forms there is a message area that reports the result of each action (for example, "Created item 3." or "Item 7 not found").

7. API

The server exposes the following API, which always returns JSON. The base URL is <http://localhost:3000>.

Method	Route	Description	Responses
GET	/api/items	Returns the full list of items.	200 list · 500 error reading the file
GET	/api/items/:id	Returns one item by its ID.	200 item · 404 not found
POST	/api/items	Creates an item. The ID is assigned as the highest existing ID + 1.	201 created item · 400 name missing
PUT	/api/items/:id	Changes an item's name.	200 item · 404 not found · 400 name missing
DELETE	/api/items/:id	Deletes an item.	200 item · 404 not found
POST	/api/reset	Restores the sample data.	200 restored list

8. How Docker is used in this project

Dockerfile

```
FROM node:18
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD [ "npm", "start" ]
```

- **FROM node:18:** starts from the official Node.js 18 image.
- **WORKDIR:** sets /usr/src/app as the working directory inside the container.
- **COPY package*.json + RUN npm install:** copies the dependency files first and installs them, so Docker can reuse this only when the code changes.
- **COPY . .:** copies the rest of the project (respecting .dockerignore).
- **EXPOSE 3000 and CMD:** documents the port and starts the application with npm start.

docker-compose.yml

```
services:
  app:
    build: .
    container_name: node-express
    ports:
      - "3000:3000"
    volumes:
      - ../usr/src/app
      - /usr/src/app/node_modules
```

- **build: .** builds the image from the Dockerfile in the current folder.
- **ports 3000:3000:** connects port 3000 on your machine to port 3000 in the container.
- **../usr/src/app:** mounts your project folder inside the container, so whatever is saved to data.json ends up on your machine.
- **/usr/src/app/node_modules:** a volume that protects the dependencies installed in the image so the previous mount does not replace them.

9. Data persistence

Items are stored in data.json. Because the project folder is mounted into the container, changes made from the web page or the API are written to the real data.json in your folder and are kept when you stop and restart the container.

This file changes as you use the application, so Git will show it as modified. To return to the initial state, use the "Reset" button on the web page.